Reliability

Reliability vs Related Concepts

People often confuse reliability with availability and fault tolerance. Here is how they differ:

Concept	Question It Answers	Example
Availability	Is the system responding?	System returns HTTP 200
Reliability	Is the response correct?	The balance returned is accurate
Fault Tolerance	Does it keep working when components fail?	Works with one database replica down
Durability	Is data preserved despite failures?	Data survives disk failure

A payment system that charges customers twice is available (it processes requests) but unreliable (it processes them incorrectly). A database that loses writes during failover is fault-tolerant (it continues operating) but not durable (data was lost).

Measuring Reliability

You cannot improve what you do not measure. Reliability has several well-established metrics that help you understand how your system behaves over time.

1. Mean Time Between Failures (MTBF)

MTBF measures the average time between failures. A higher MTBF means failures are less frequent.

`MTBF = Total Operating Time / Number of Failures`

Example:

```
- System ran for 10,000 hours
- Experienced 5 failures
- MTBF = 10,000 / 5 = 2,000 hours
```

An MTBF of 2,000 hours means you can expect one failure roughly every 83 days. This helps with planning. If you have 100 servers each with MTBF of 2,000 hours, you will see approximately one server failure per day across your fleet.

2. Mean Time To Recovery (MTTR)

MTTR measures how long it takes to restore the system after a failure. A lower MTTR means faster recovery.

`MTTR = Total Downtime / Number of Failures`

Example:

```
- 5 failures occurred
- Total repair time: 10 hours
- MTTR = 10 / 5 = 2 hours per failure
```

MTTR includes detection time, diagnosis time, repair time, and verification time. Reducing MTTR often has more impact than reducing failure rate. If you cannot prevent failures, at least recover quickly.

Part 1: The Single Server (83 Days)

If one server has an MTBF of 2,000 hours, it means you can expect it to run for 2,000 hours before a component (like a hard drive or power supply) dies.

To convert those hours into days, you just divide by 24:

- $2,000 \text{ hours} \div 24 \text{ hours per day} = \mathbf{83.33} \text{ days}$

So, a single server running continuously will hit its 2,000-hour limit in roughly 83 days.

Part 2: The Fleet of 100 Servers (~1 Failure per Day)

When you have multiple servers, they are all running and aging at the exact same time. They are collectively generating "machine-hours" much faster than the clock on your wall.

- If you have **1 server**, it generates **1 machine-hour** every hour.
- If you have **100 servers**, your fleet generates **100 machine-hours** every hour.

To find out how many failures you will get in a single day, you calculate the total machine-hours your fleet generates in 24 hours:

- $100 \text{ servers} \times 24 \text{ hours} = \mathbf{2,400} \text{ machine-hours per day}$

Since we know a failure happens every 2,000 machine-hours, we divide the daily total by the MTBF:

- $2,400 \text{ machine-hours} \div 2,000 \text{ MTBF} = \mathbf{1.2} \text{ failures per day}$

(Hence, "approximately one server failure per day" across the fleet).

The Airplane Analogy

Imagine you are flying across the ocean.

- **A Single-Engine Plane:** The engine is highly reliable. It only fails once every 10,000 hours. But if it *does* fail, the plane crashes. Your "system" (the plane) has a 100% dependency on that single component.
- **A Four-Engine Plane:** Because you have four engines, you will experience an engine failure four times as often (every 2,500 hours). You will have to do way more maintenance. **However**, the plane can fly perfectly fine on just two or three engines.

Even though the *components* fail more frequently, the *system* almost never crashes, because it has **Redundancy**.

3. Error Rate

Percentage of requests that result in errors.

```
Error Rate = Failed Requests / Total Requests × 100%
```

System Type	Target Error Rate	Meaning
Critical systems	< 0.01%	1 in 10,000 requests fails
Standard systems	< 0.1%	1 in 1,000 requests fails
Tolerant systems	< 1%	1 in 100 requests fails

4. Data Correctness

Percentage of responses that contain correct data.

```
Correctness = Correct Responses / Total Responses × 100%
```

This is the often-overlooked metric. A system can have 99.99% availability and 0.01% error rate, but if 1% of successful responses contain wrong data, you have a reliability problem. Users received a response, it just was not the right one.

Key Principles of Reliable Systems

To build reliable systems, engineers typically focus on several core principles:

Redundancy

Redundancy means having backup components ready to take over if one part fails. This could involve multiple servers, duplicate network paths, or backup databases.

Failover Mechanisms

Failover is the process by which a system automatically switches to a redundant or standby component when a failure is detected. This ensures continuous operation without noticeable disruption to users.

Load Balancing

Load balancing distributes incoming traffic across multiple servers. This not only improves performance but also prevents any single server from becoming a single point of failure.

Monitoring and Alerting

A reliable system is constantly monitored. Tools and dashboards track system health and performance, while alerting mechanisms notify engineers of issues before they escalate into major problems.

Graceful Degradation

Even when parts of the system fail, a well-designed system can still provide core functionality rather than going completely offline. This concept is known as graceful degradation.

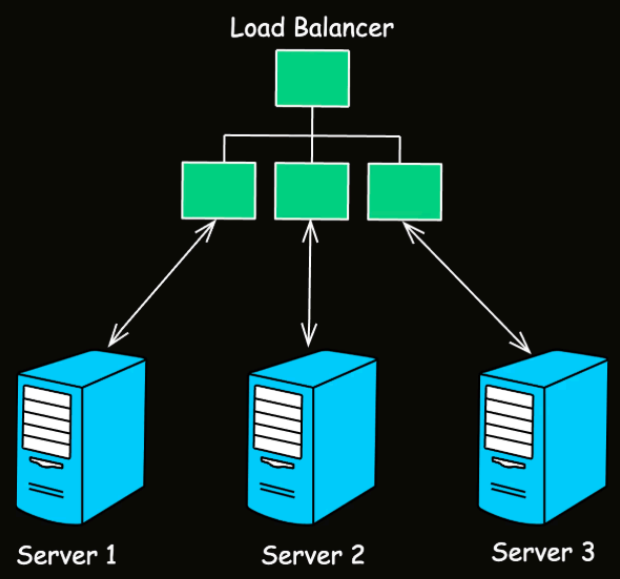
Techniques to Enhance Reliability

Now that we understand the principles, let's look at some practical techniques to implement reliability in your systems.

1. Redundant Architectures

The most fundamental reliability technique is having more components than you need. If one fails, others continue operating.

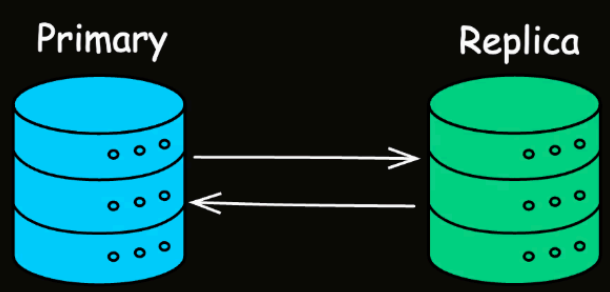
For example, if you have a web server handling user requests, deploy several servers behind a load balancer:



If one server fails, the load balancer automatically routes traffic to the remaining servers.

2. Data Replication

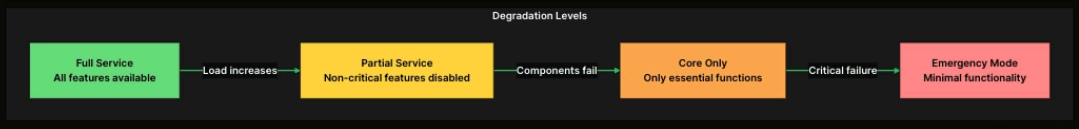
Ensure your data is not stored in a single location. Use data replication strategies across multiple databases or data centers.



This way, if one database fails, the system can still access a copy from another location.

3. Graceful Degradation

When parts of the system fail, graceful degradation keeps the core functionality working. Instead of complete failure, the system provides reduced service.



Consider an e-commerce site:

- **Full service:** Personalized recommendations, real-time inventory, all payment options
- **Partial service:** Generic recommendations, cached inventory, primary payment options
- **Core only:** Browse products, checkout with basic payment
- **Emergency mode:** Display cached product pages, accept orders for later processing

"An escalator can never break: it can only become stairs.”

How it Works in Software

In modern system design, applications are broken down into dozens of smaller, independent pieces called microservices. To achieve graceful degradation, engineers divide these services into two categories: **Core** and **Non-Core**.

Here are three common examples of how this looks in practice:

1. E-Commerce (Amazon/Flipkart)

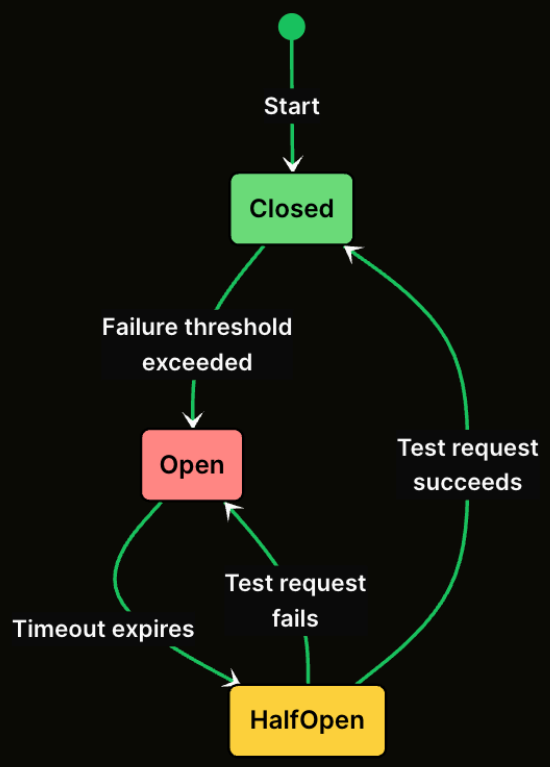
- **Core:** Viewing a product, adding it to the cart, and paying.
- **Non-Core:** The "Customers who bought this also bought..." recommendation engine.
- **The Degradation:** If the recommendation database crashes, a circuit breaker instantly cuts the connection. Instead of the whole webpage failing to load because it's waiting for recommendations, the webpage simply loads *without* that specific row of products. The user can still buy the item, and the business still makes money.

2. Video Streaming (Netflix/YouTube)

- **Core:** Playing the video.
- **Non-Core:** Playing the video in 4K resolution.
- **The Degradation:** If your network connection suddenly drops in quality, Netflix doesn't crash or throw an error. It gracefully degrades the video quality from 4K to 1080p, and then down to 480p, prioritizing uninterrupted playback over visual fidelity.

4. Circuit Breakers

In a microservices architecture, one service failing can cascade failures throughout the system. Circuit breakers detect when a service is failing and temporarily cut off requests to prevent overload, allowing the system to recover gracefully.



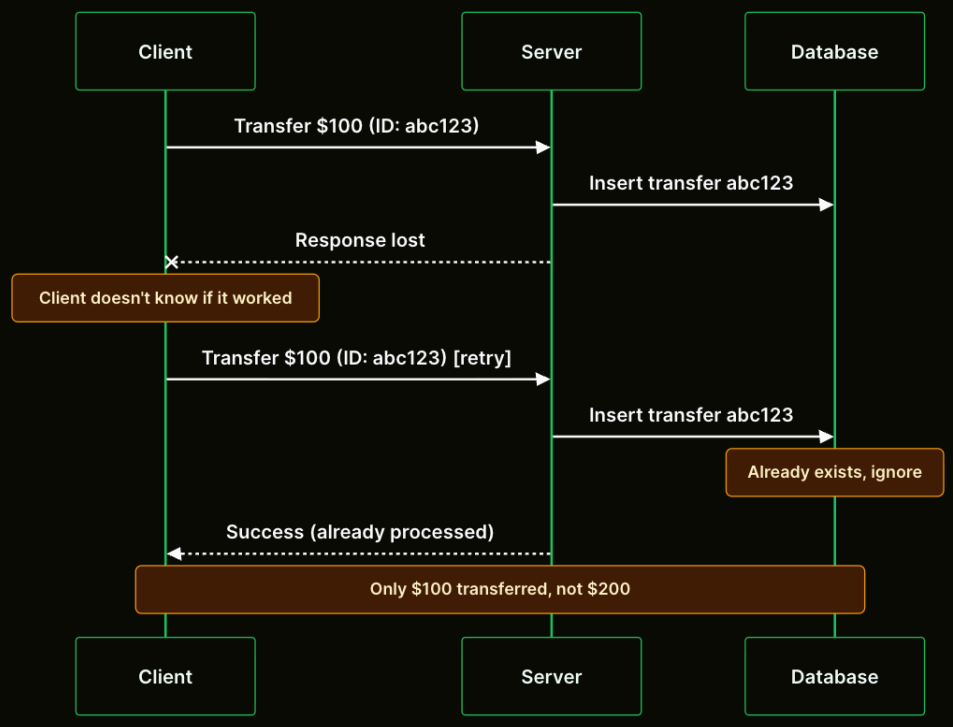
In the **closed** state, requests pass through normally. Failures are counted. When failures exceed a threshold (e.g., 5 failures in 30 seconds), the circuit opens.

In the **open** state, requests fail immediately without calling the dependency. This prevents wasting resources and allows the dependency time to recover.

After a timeout (e.g., 30 seconds), the circuit moves to **half-open**. A limited number of test requests are allowed through. If they succeed, the circuit closes. If they fail, it opens again.

5. Idempotency

Network failures make it unclear whether a request succeeded or failed. If you retry, you might execute the operation twice. Idempotent operations produce the same result regardless of how many times they are executed.



The idempotency key (abc123) allows the server to detect retries. The first execution stores the key. Subsequent executions with the same key return the stored result without re-executing.

Stripe, PayPal, and other payment processors require idempotency keys for money-moving operations. Without them, network issues could cause duplicate charges.

If guardrails like Circuit Breakers and Message Queues are designed to handle system failures, Idempotency is designed to handle **user impatience** and **network hiccups**.

The Problem: The "Double Charge" Nightmare

Imagine a user is buying a $50 shirt on your app.

1. The user taps "Pay Now".
2. Your app sends a request to the server: `CHARGE_CARD($50)`.
3. The server processes the charge successfully.
4. **The Failure:** Before the server can send the "Success!" message back to the phone, the user's cell signal drops for a second.
5. The app never gets the success message. It assumes the request failed.
6. The app automatically **retries** the request (or the user gets impatient and taps "Pay Now" again).
7. The server receives a second `CHARGE_CARD($50)` request and processes it.

The user just paid $100 for a $50 shirt. This is a non-idempotent system, and it will destroy user trust instantly.

The Real-World Analogy

- **Not Idempotent (Like adding salt to soup):** If you add a pinch of salt to a soup, it gets saltier. If you forget whether you added it and add another pinch, it gets *too* salty. The state changes every time you perform the action.
- **Idempotent (Like an elevator button):** When you walk up to an elevator, you press the "Up" button. It lights up. If you are in a rush and mash the button 10 more times, the elevator doesn't arrive 10 times, nor does it go up 10 times faster. The end result of pressing it 11 times is exactly the same as pressing it once.

Question:

To generate a unique, random ID for every distinct user action.

so whenever user taps pay now twice , wouldnt this generate two different keys?

Answer:

You are absolutely right. If the mobile app generates a brand new random key every single time the user's finger physically touches the "Pay Now" button, the server will see two different keys, treat them as two entirely different orders, and charge the user twice. The idempotency pattern would completely fail.

To prevent this, engineers have to be very careful about **when** they generate the key. The secret is that the key must be tied to the **Intent of the Transaction** (the specific cart state), not the physical button click.

Here is how it is actually implemented to avoid the trap you just described:

The Correct Implementation: Tying the Key to the Session

Instead of generating the key at the moment of the click, the app generates it earlier and holds onto it.

1. **The Setup:** The user finishes shopping and navigates to the "Checkout / Review Order" screen.
2. **Key Generation:** The moment that screen loads, the app generates the Idempotency Key (e.g., `Cart_XYZ_Payment`) and stores it temporarily in the phone's memory.
3. **Attempt 1:** The user taps "Pay Now". The app sends the request using the `Cart_XYZ_Payment` key.
4. **The Glitch:** The network stalls. The button stops spinning, and the user gets impatient.
5. **Attempt 2:** The user taps "Pay Now" again. Because the user is still on the exact same checkout screen trying to pay for the exact same cart, the app **reuses the existing key** in its memory. It sends a second request, but *still* uses `Cart_XYZ_Payment`.
6. **The Server:** The server receives both requests, sees the identical keys, and safely ignores the second one.

When does the key actually change?

The app will only throw away that key and generate a new one if the underlying "intent" changes. For example:

- The user taps the "Back" arrow, adds another item to their cart, and returns to the checkout screen.
- The user successfully pays, gets a receipt, and then starts a brand new shopping session.
- A strict timer expires (e.g., the key is only valid for 24 hours).

The Backend Alternative (Server-Side Keys)

Sometimes, you can't trust the mobile app (the client) to manage keys correctly. In these cases, the server dictates the key based on the data itself.

Instead of a random string, the server creates a unique "hash" (a digital fingerprint) out of the order details: `Hash(User ID + Cart Items + Total Price + Timestamp of Checkout Creation)`.

If the user spams the "Pay Now" button, every single request will contain the exact same data payload. The server calculates the hash for each incoming request. Because the data hasn't changed, the hash is identical every time, and the server knows it's a duplicate request.